DBMS LAB ASSIGNMENT-4

Name: Titli Sadhukhan
Roll No: UG/SOET/29/24/076

SQL statements are for MySQL Workbench. Outputs are expected results in text form; exact formatting may vary by version.

1. Create Customer and Movie tables with the given constraints and insert the supplied records.

```
CREATE DATABASE MovieDB;
USE MovieDB;

CREATE TABLE Customer (
  Cust_id CHAR(3) PRIMARY KEY,
  Fname VARCHAR(20) NOT NULL,
  Lname VARCHAR(20),
  Area CHAR(2),
  Phone BIGINT
);

INSERT INTO Customer VALUES
('A01','Ivan','Ross','SA',6125467),
('A02','Vandana','Ray','MU',5560379),
('A03','Pramada','Jauguste','DA',4560389),
('A04','Basu','Navindi','BA',6125401),
('A05','Ravi','Shridhar','NA',NULL),
('A06','Rukmini','Aiyer','GH',5125274);

CREATE TABLE Movie (
  Mv_no INT PRIMARY KEY,
  Cust_id CHAR(3),
  Title VARCHAR(30) NOT NULL,
  Star CHAR(2) NOT NULL,
  Price INT CHECK (Price BETWEEN 100 AND 250),
  CONSTRAINT fk_movie_customer
    FOREIGN KEY (Cust_id) REFERENCES Customer(Cust_id)
);

INSERT INTO Movie VALUES
(1,'A02','Bloody','JC',181),
(2,'A04','The Firm','TC',200),
(3,'A01','Pretty Woman','RG',151),
(4,'A06','Home Alone','MC',150),
(5,'A05','The Fugitive','MF',200),
(6,'A03','Coma','MD',100),
(7,'A02','Dracula','GO',150),
(8,'A06','Quick Change','BM',100),
(9,'A03','Gone with the Wind','CB',200),
(10,'A05','Carry on Doctor','LP',100);

SELECT * FROM Customer;
SELECT * FROM Movie;
```

Expected result: 6 customer rows and 10 movie rows inserted.
Note: Phone is nullable because the supplied data contains NULL for A05. MySQL enforces CHECK constraints from version 8.0.16 onward.

2. Prove entity integrity for both tables.

```
SELECT * FROM Customer WHERE Cust_id IS NULL;
SELECT * FROM Movie WHERE Mv_no IS NULL;
SELECT Cust_id, COUNT(*) FROM Customer GROUP BY Cust_id HAVING COUNT(*) > 1;
SELECT Mv_no, COUNT(*) FROM Movie GROUP BY Mv_no HAVING COUNT(*) > 1;
```

Expected output: Empty set for all four queries. Primary keys cannot contain NULL or duplicate values.

3. Prove referential integrity.

```
SELECT m.Cust_id
FROM Movie AS m
LEFT JOIN Customer AS c ON m.Cust_id = c.Cust_id
WHERE c.Cust_id IS NULL;
```

Expected output: Empty set. Every Movie.Cust_id matches a Customer.Cust_id.

4. Prove domain integrity for Movie.

```
SELECT * FROM Movie
WHERE Price < 100 OR Price > 250;
```

Expected output: Empty set. All supplied prices are between 100 and 250.

5. Display movie titles whose price is greater than 100 and less than 200.

```
SELECT Title FROM Movie
WHERE Price > 100 AND Price < 200;
```

Expected output:
Bloody
Pretty Woman
Home Alone
Dracula
4 rows in set.

6. Display Cust_id for customers who have seen movies starring JC, TC, or MC.

```
SELECT Cust_id FROM Movie
WHERE Star IN ('JC', 'TC', 'MC');
```

Expected output:
A02
A04
A06
3 rows in set.

7. Display customer details where Area contains the letter A.

```
SELECT * FROM Customer
WHERE Area LIKE '%A%';
```

Expected output: A01 Ivan Ross SA 6125467; A03 Pramada Jauguste DA 4560389; A04 Basu Navindi BA 6125401; A05 Ravi Shridhar NA NULL.

8. Display movie titles with price within 180 and title length exactly 6 characters.

```
SELECT Title FROM Movie
WHERE Price <= 180 AND CHAR_LENGTH(Title) = 6;
```

Expected output: Empty set. Here, “within 180” is interpreted as Price <= 180.

9. Display movie names, original prices, and prices after a 10% increase using an alias.

```
SELECT Title,
       Price AS Original_Price,
       Price * 1.10 AS Increased_Price
FROM Movie;
```

Expected increased prices:
Bloody 199.10; The Firm 220.00; Pretty Woman 166.10; Home Alone 165.00;
The Fugitive 220.00; Coma 110.00; Dracula 165.00; Quick Change 110.00;
Gone with the Wind 220.00; Carry on Doctor 110.00.

10. Display customer details in the sentence format shown in the assignment.

```
SELECT CONCAT(Fname, ' ', Lname,
  ' stays in ', Area,
  ' and his phone number is ',
  COALESCE(CAST(Phone AS CHAR), 'NULL'), '.')
  AS Customer_Details
FROM Customer;
```

Expected output: One sentence per customer. First row: “Ivan Ross stays in SA and his phone number is 6125467.” Ravi Shridhar displays NULL for the phone.

11. Add a NOT NULL constraint to Customer.Lname.

```
ALTER TABLE Customer
MODIFY Lname VARCHAR(20) NOT NULL;
```

Expected output: Query OK, 0 rows affected. Existing Lname values are non-NULL.

12. Display the full name of the customer whose phone number is missing.

```
SELECT CONCAT(Fname, ' ', Lname) AS Customer_Name
FROM Customer
WHERE Phone IS NULL;
```

Expected output: Ravi Shridhar (1 row).

13. Add a phone number for the customer identified in Q7 whose phone is missing.

```
UPDATE Customer
SET Phone = 9876543210
WHERE Cust_id = 'A05';

SELECT * FROM Customer WHERE Cust_id = 'A05';
```

Expected output: Query OK, 1 row affected. A05 Ravi Shridhar now displays phone 9876543210. This is an example number; replace it if a specific number was assigned.

14. Display unique Cust_id values from Movie.

```
SELECT DISTINCT Cust_id
FROM Movie
ORDER BY Cust_id;
```

Expected output: A01, A02, A03, A04, A05, A06 (6 rows).

15. Remove the NOT NULL constraint from Movie.Star.

```
ALTER TABLE Movie
MODIFY Star CHAR(2) NULL;
```

Expected output: Query OK, 0 rows affected.

16. Delete any one row from Customer; show the reason if deletion is not possible.

```
DELETE FROM Customer
WHERE Cust_id = 'A01';
```

Expected output: Deletion is rejected because A01 is referenced in Movie (Mv_no 3). Typical MySQL error 1451: foreign key constraint fails.

17. Delete any one row from Movie.

```
DELETE FROM Movie WHERE Mv_no = 10;
SELECT * FROM Movie WHERE Mv_no = 10;
```

Expected output: Query OK, 1 row affected; verification SELECT returns Empty set.

18. Drop Customer; show the reason if dropping is not possible.

```
DROP TABLE Customer;
```

Expected output: Rejected while Movie has a foreign key referencing Customer. Typical MySQL error 3730: the table is referenced by a foreign key constraint.

19. Drop the Movie table.

```
DROP TABLE Movie;
```

Expected output: Query OK. To complete both Q19 and Q20, perform Q20 first; otherwise dropping Movie means there is no table left for Q20.

20. Drop the foreign key constraint from Movie.

```
SHOW CREATE TABLE Movie;

-- Use the exact foreign-key name shown above.
ALTER TABLE Movie
DROP FOREIGN KEY fk_movie_customer;
```

Expected output: Query OK. The constraint name may differ if your table was created differently; use the name shown by SHOW CREATE TABLE. Run Q20 before Q19.